

#Blog Project - API Documentation

📄 Table of Contents

-Project Overview

-Project Architecture

-Database Models

-API Endpoints

-Installation & Setup

-Usage Examples

##Project Overview

###Description

This is a **Django-based Blog Application** that provides a complete blogging platform with user-friendly features. The application allows users to:

- View published blog posts
- Browse posts by categories
- Read detailed blog articles
- Add comments and replies to blog posts
- Filter posts by section (Popular, Recent, Trending)

###Key Features

- ✅ **Multi-category blog posts** - Organize content by categories
- ✅ **Rich text editing** - Support for formatted content using CKEditor
- ✅ **Comment system** - Nested comments with reply functionality
- ✅ **Post management** - Draft and publish workflow
- ✅ **Image uploads** - Blog post featured images
- ✅ **Responsive design** - Bootstrap-based responsive UI

✓ **SEO-friendly** - Auto-generated slugs for clean URLs

###Technology Stack

-Framework: Django 5.0.7

-Database: PostgreSQL (with SQLAlchemy ORM support)

-Web Server: Gunicorn 23.0.0

-File Storage: Cloudinary integration for media files

-Rich Text Editor: Django-CKEditor

-Templating: Django Templates (HTML/CSS/Bootstrap)

-Frontend: Bootstrap, Font Awesome, HTML5/CSS3

##Project Architecture

###Directory Structure

```
Blog_project/
├── blog/                # Main Django project configuration
│   ├── settings.py      # Project settings and configuration
│   ├── urls.py          # Main URL router
│   ├── wsgi.py          # WSGI application entry point
│   ├── asgi.py          # ASGI application entry point
│   └── storage.py        # Cloudinary storage configuration
│
├── home/                # Django app containing blog functionality
│   ├── models.py        # Database models (Blog, Category, Comment)
│   ├── views.py         # View functions handling requests
│   ├── urls.py          # App-level URL routing
│   ├── admin.py         # Django admin configuration
│   └── migrations/      # Database migrations
```

```
|
|
├─ templates/                # HTML templates
|   └─ index.html            # Homepage
|   └─ blog_detail.html      # Blog post detail page
|   └─ category.html         # Category listing page
|   └─ ... (other templates)
|
|
├─ static/                   # Static files (CSS, JS, images)
├─ Images/                   # User-uploaded images
├─ manage.py                 # Django management command utility
├─ requirements.txt          # Python dependencies
└─ README.md                 # Project README
```

###Data Flow Architecture

```
User Request
    ↓
Django URL Router (blog/urls.py → home/urls.py)
    ↓
View Function (home/views.py)
    ↓
Query Database Models (home/models.py)
    ↓
Render Template (templates/)
    ↓
HTTP Response to User
```

##Database Models

###1. Category Model

Represents blog post categories for content organization.

```
classCategory(models.Model):  
  
    id            :AutoField (Primary Key)  
  
    name          :CharField(max_length=150)- Category name  
  
    slug          :AutoSlugField(unique=True)- URL-friendly slug
```

Relationships:

- One-to-Many: Category → Blog (related_name='blogs')

Example Data:

```
ID: 1, Name: "Technology", Slug: "technology"  
  
ID: 2, Name: "Lifestyle", Slug: "lifestyle"
```

###2. Blog Model

Represents individual blog posts.

```
classBlog(models.Model):  
  
    id            :AutoField (Primary Key)  
  
    title         :CharField(max_length=200)- Post title  
  
    author        :CharField(max_length=100,default='Admin')- Author name  
  
    img           :ImageField(upload_to='Images')- Featured image  
  
    content       :RichTextField()- Post content (HTML-enabled)  
  
    category      :ForeignKey(Category)- Associated category
```

```
blog_slug      :AutoSlugField(unique=True)- URL-friendly slug
date           :DateField(auto_now_add=True)- Publication date
status         :CharField(choices=['0'=DRAFT,'1'=PUBLISH])- Post status
section        :CharField(choices=['Popular','Recent','Trending'])- Section
Main_post      :BooleanField(default=False)- Featured post flag
```

Status Values:

- '0' - DRAFT (not visible to public)

- '1' - PUBLISH (visible to public)

Section Values:

- 'Popular' - Popular posts section

- 'Recent' - Recent posts section

- 'Trending' - Trending posts section

Relationships:

- Foreign Key: Category (on_delete=CASCADE)
- One-to-Many: Blog → Comment (related_name='comments')

Default Ordering: By date (newest first)

###3. Comment Model

Represents comments and nested replies on blog posts.

```
class Comment(models.Model):
    id          :AutoField (Primary Key)
    post        :ForeignKey(Blog)- Associated blog post
    blog_id     : IntegerField - Cached blog post ID
    name        :CharField(max_length=100)- Commenter name
```

```
email          :EmailField()- Commenter email

website        :URLField(blank=True,null=True)- Commenter website

comment        :TextField()- Comment content

date           : DateTimeField - Comment timestamp

parent         :ForeignKey(self)- Parent comment for nested replies
```

Features:

- ☒ Nested comments (replies to comments)
- ☒ Self-referential foreign key for threading
- ☒ Email validation
- ☒ Optional website field

Default Ordering: By date (newest first)

##API Endpoints

###1. Homepage / Blog Listing

Endpoint:GET /

View Function:index(request)

Template:index.html

Description: Displays the homepage with all published blog posts, featured post, recent news, and categories.

Query Parameters: None

Response Data (Context):

```
{

    'post': QuerySet[Blog]          # All blog posts ordered by newest first

    'recent_news': QuerySet[Blog]  # Recent section posts (max 5)

    'category': QuerySet[Category] # All categories
```

```
'main_post': QuerySet[Blog]      # Featured blog post (max 1)

'blog_cat': QuerySet[Category]  # All categories for sidebar

}
```

HTTP Status Codes:

-200 OK - Successfully retrieved homepage

Example URL:

```
GET http://127.0.0.1:8000/
```

###2. Blog Post Detail

Endpoint:GET /blog/<slug:slug>/

View Function:blog_detail(request, slug)

Template:blog_detail.html

Description: Displays a single blog post with full content, comments, and related posts.

URL Parameters:

Parameter	Type	Required	Description
-----------	------	----------	-------------

----- ----- ----- -----

slug	string	✔ Yes	URL-friendly slug of the blog post
------	--------	-------	------------------------------------

Response Data (Context):

```
{

    'post': Blog                  # The blog post object

}
```

```

'share_url': string                # Full absolute URL for sharing

'posts': QuerySet[Blog]            # Recent posts (max 5)

'category': QuerySet[Category]     # All categories

'recent_news': QuerySet[Blog]      # Recent section posts (max 5)

'comments': QuerySet[Comment]     # Root-level comments (parent=None)

}

```

HTTP Status Codes:

-200 OK - Post found and displayed

-404 Not Found - Blog post slug doesn't exist

Example URLs:

```

GET http://127.0.0.1:8000/blog/my-first-blog-post/

GET http://127.0.0.1:8000/blog/django-tutorial/

```

###3. Category Listing

Endpoint: GET /category/<str:slug>/

View Function: category(request, slug)

Template: category.html

Description: Displays all blog posts in a specific category.

URL Parameters:

Parameter	Type	Required	Description
-----------	------	----------	-------------

slug	string	Yes	URL-friendly slug of the category
------	--------	-----	-----------------------------------

Response Data (Context):


```
{
    'cat': QuerySet[Category]          # All categories for navigation
    'active_category': string          # Current category slug
    'blog_cat': QuerySet[Blog]         # Blog posts in this category
    'category_obj': Category           # The category object
}
```

HTTP Status Codes:

-200 OK - Category found and posts displayed

-404 Not Found - Category slug doesn't exist

Example URLs:

```
GET http://127.0.0.1:8000/category/technology/
```

```
GET http://127.0.0.1:8000/category/lifestyle/
```

###4. Add Comment

Endpoint: POST /blog/<slug:slug>/add_comment/

View Function: add_comment(request, slug)

Redirect Target: blog_detail page

Description: Adds a new comment or reply to a blog post. Accepts both POST requests and redirects GET requests.

URL Parameters:

Parameter	Type	Required	Description
-----------	------	----------	-------------

--	--	--	--

|slug| string | ☒ Yes | URL-friendly slug of the blog post |

Request Method:POST

Form Parameters (POST Data):

| Parameter | Type | Required | Description |

|-----|-----|-----|-----|

|InputName| string | ☒ Yes | Commenter's name (max 100 chars) |

|inputEmail| email | ☒ Yes | Commenter's email address |

|InputWeb| URL | ☐ No | Commenter's website (optional) |

|InputComment| text | ☒ Yes | Comment content |

|parent_id| integer | ☐ No | Parent comment ID for nested replies |

Response:

-**Success:** Redirects to the blog post detail page

-**Error:** Redirects to homepage if blog post not found

HTTP Status Codes:

-302 Found - Comment added, redirect to blog detail

-302 Found - Invalid request, redirect to homepage

-404 Not Found - Blog post slug doesn't exist

Example Request:

```
POSThttp://127.0.0.1:8000/blog/my-first-blog-post/add_comment/

FormData:

InputName=JohnDoe

inputEmail=john@example.com

InputWeb=https://johndoe.com
```

```
InputComment=Greatpost!Veryinformative.
```

```
parent_id=(emptyforrootcomment)
```

Example Reply Request:

```
POSThttp://127.0.0.1:8000/blog/my-first-blog-post/add_comment/
```

```
FormData:
```

```
InputName=JaneSmith
```

```
InputEmail=jane@example.com
```

```
InputWeb=(empty)
```

```
InputComment=ThanksforthecommentJohn!
```

```
parent_id=5
```

##Installation & Setup

###Prerequisites

- Python 3.8+
- pip (Python package manager)
- PostgreSQL database (recommended for production)

###Step 1: Clone the Repository

```
gitclone<repository-url>
```

```
cdBlog_project
```

###Step 2: Create Virtual Environment (Recommended)

```
python-mvenvvenv
```

```
# Windows
```

```
venv\Scripts\activate
```

```
# macOS/Linux
```

```
sourcevenv/bin/activate
```

###Step 3: Install Dependencies

```
pipinstall-rrequirements.txt
```

###Step 4: Configure Database

Update `blog/settings.py` with your database credentials:

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.postgresql',  
        'NAME': 'your_database_name',  
        'USER': 'your_username',  
        'PASSWORD': 'your_password',  
        'HOST': 'localhost',  
        'PORT': '5432',  
    }  
}
```

###Step 5: Run Migrations

```
pythonmanage.pymigrate
```

###Step 6: Create Superuser (Admin Account)

```
pythonmanage.pycratesuperuser
```

###Step 7: Run Development Server

```
pythonmanage.pyrunserver
```

Access the application at: **http://127.0.0.1:8000/**

###Admin Dashboard

Access at: **http://127.0.0.1:8000/admin/**

##Usage Examples

###1. View Homepage

```
curlhttp://127.0.0.1:8000/
```

Returns an HTML page with all blog posts, featured posts, and categories.

###2. View a Specific Blog Post

```
curlhttp://127.0.0.1:8000/blog/django-tutorial/
```

Returns the blog post detail page with comments and related posts.

###3. Browse by Category

```
curlhttp://127.0.0.1:8000/category/technology/
```

Returns all blog posts in the "Technology" category.

###4. Submit a Comment via Form (HTML)

```
<formmethod="POST"action="/blog/django-tutorial/add_comment/">

  {% csrf_token %}

  <inputtype="text"name="InputName"placeholder="Your Name"required>

  <inputtype="email"name="InputEmail"placeholder="Your Email"required>

  <inputtype="url"name="InputWeb"placeholder="Your Website (Optional)">

  <textareaname="InputComment"placeholder="Your Comment"required></textarea>

  <buttontype="submit">Submit Comment</button>

</form>
```

###5. Submit a Reply to a Comment

```
<formmethod="POST"action="/blog/django-tutorial/add_comment/">

  {% csrf_token %}

  <inputtype="text"name="InputName"placeholder="Your Name"required>

  <inputtype="email"name="InputEmail"placeholder="Your Email"required>

  <inputtype="url"name="InputWeb"placeholder="Your Website (Optional)">

  <textareaname="InputComment"placeholder="Your Reply"required></textarea>
```

```
<input type="hidden" name="parent_id" value="5"><!-- ID of parent comment -->

<button type="submit">Submit Reply</button>

</form>
```

###6. Django Admin Operations

```
# Create a new category

pythonmanage.pyshell

>>> from home.models import Category

>>> Category.objects.create(name="Python Programming")

# Create a blog post

>>> from home.models import Blog, Category

>>> cat = Category.objects.get(name="Python Programming")

>>> Blog.objects.create(
...     title="Intro to Django",
...     author="Admin",
...     content="<p>Django is awesome!</p>",
...     category=cat,
...     section="Recent",
...     status="1"
... )

# Query all published posts

>>> Blog.objects.filter(status='1')

# Query comments on a post

>>> from home.models import Blog

>>> post = Blog.objects.get(blog_slug="intro-to-django")
```

```
>>> post.comments.all()
```

##API Response Format

###Success Response (Blog Post)

```
HTTP 200 OK
```

```
Content-Type: text/html
```

```
{
  "post":{
    "id":1,
    "title":"My First Blog Post",
    "author":"Admin",
    "content":"<p>This is my first blog post...</p>",
    "category":{
      "id":1,
      "name":"Technology",
      "slug":"technology"
    },
    "blog_slug":"my-first-blog-post",
    "date":"2024-01-15",
    "status":"1",
    "section":"Recent",
    "Main_post":false
  }
}
```

###Comment Response


```
{  
  "id":1,  
  "name":"John Doe",  
  "email":"john@example.com",  
  "website":"https://johndoe.com",  
  "comment":"Great post!",  
  "date":"2024-01-15T10:30:00Z",  
  "post":1,  
  "parent":null,  
  "replies":[  
    {  
      "id":2,  
      "name":"Jane Smith",  
      "email":"jane@example.com",  
      "comment":"Thanks for the comment John!",  
      "date":"2024-01-15T11:00:00Z",  
      "parent":1  
    }  
  ]  
}
```

###Error Response (404)

HTTP 404 Not Found

Content-Type: text/html

404: Page Not Found

The blog post or category you're looking for doesn't exist.

##Important Notes

###Security Considerations

⚠️ **CSRF Protection:** All POST requests require Django CSRF token (`{% csrf_token %}`)

⚠️ **Email Validation:** Comments require valid email addresses

⚠️ **URL Validation:** Website field is optional but must be valid URL if provided

⚠️ **Content Sanitization:** Use Django's built-in templating for XSS protection

###Performance Tips

Database Optimization:

- Posts are ordered by date (newest first)
- Use Django's `select_related()` for category queries
- Implement pagination for large post collections

Media Storage:

- Images are uploaded to Cloudinary (cloud-based storage)
- Configure CloudinaryStorage in `blog/storage.py`

###File Upload Limits

-**Image Format:** JPG, PNG, GIF, WebP supported

-**Max Size:** Configured in settings (default: 5MB)

-**Upload Directory:** `Images/` folder

##Deployment

###Using Gunicorn

```
pipinstallgunicorn

gunicornblog.wsgi--bind0.0.0.0:8000
```

###Using WhiteNoise (Static Files)

```
pythonmanage.pycollectstatic
```

##Troubleshooting

Issue	Solution
-------	----------

-----	-----
-------	-------

404 on blog post	Check that blog status is '1' (PUBLISH)
------------------	---

Comments not showing	Ensure comments have <code>parent=None</code> for root comments
----------------------	---

Images not uploading	Configure Cloudinary storage in settings
----------------------	--

Database errors	Run <code>python manage.py migrate</code>
-----------------	---

Admin not accessible	Create superuser: <code>python manage.py createsuperuser</code>
----------------------	---

##References

- [Django Documentation](#)

- [Django-CKEditor](#)

- [Cloudinary Storage](#)

- [Bootstrap Documentation](#)

- [Font Awesome Icons](#)

Last Updated: May 15, 2026

Version: 1.0

License: MIT License